

Fast and Live Model **Auto Scaling** with **$O(1)$** Host Caching

Dingyan Zhang¹, Haotian Wang^{*1}, Yang Liu^{*1}, Xingda Wei¹, Yizhou Shan², Rong Chen¹, and Haibo Chen¹

¹Institute of Parallel and Distributed Systems, SEIEE, Shanghai Jiao Tong University

²Huawei Cloud

Background: MAAS

- Maas: model-serving-as-a-service systems
 - two key objectives:
 - Maximizing **goodput**
 - minimizing **instances**
- Challenge: **the unpredictable short-term fluctuations** in a model's hardware demands

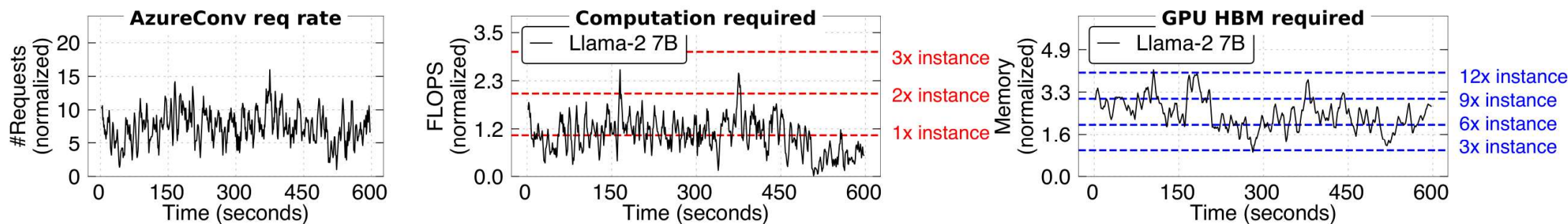


Figure 1: The timeline of request incoming rate of a real-world AzureConv [11] trace (a), its computation (b) and memory requirements (c), when serving this workload without SLO violation.

Background: Autoscaling

- dynamically deploys serving instances on spare GPUs
- bottlenecked by the data plane
 - loads model parameters into serving instances
 - **stop-the-world**: scaled instances cannot serve requests until all parameters are loaded.
- ServerlessLLM (The state-of-the-art autoscaling system)
 - uses SSDs to store the parameters and employs a parameter loading mechanism that can fully utilize SSD bandwidth to load the parameters into instances.
 - low hit rate

Online network-based scale plan generation

- Group all targets connected via NVLink into a **group**
- remove links between sources and targets that could cause **interference**
- generate the multicast plan by greedily constructing **broadcasting chains**

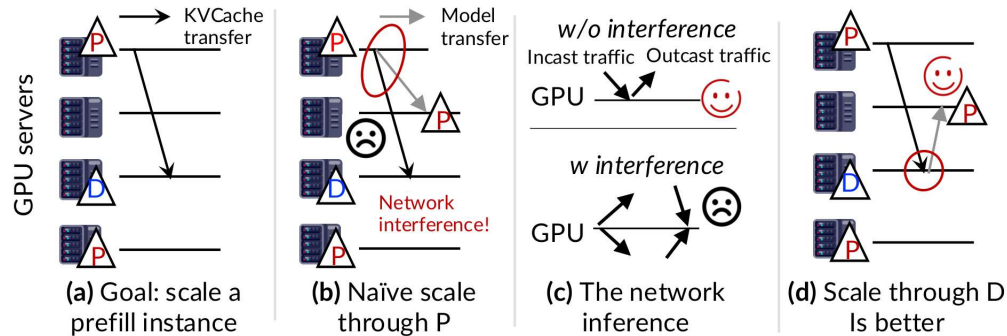


Figure 8: (a) An illustration of scaling a prefill instance for LLM P/D disaggregated serving. (b) Naïvely scaling from a prefill instance imposes network interference. (c) Interference can be avoided by leveraging the bi-directional feature of modern DCN networking. (d) An improved scale plan with the bi-directional in mind.

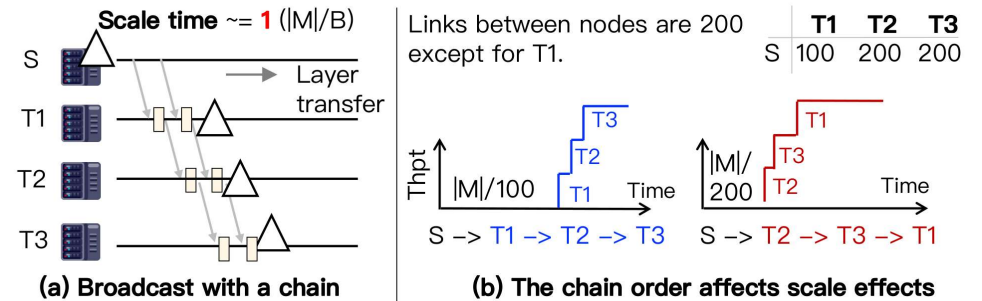


Figure 10: An illustration of (a) why chain is friendly to broadcast with huge bandwidth requirement and (b) why we choose a specific chain order. $|M|$ is the model size and B is the slowest network bandwidth between nodes in a chain.

Efficient live autoscaling with zigzag scheduling

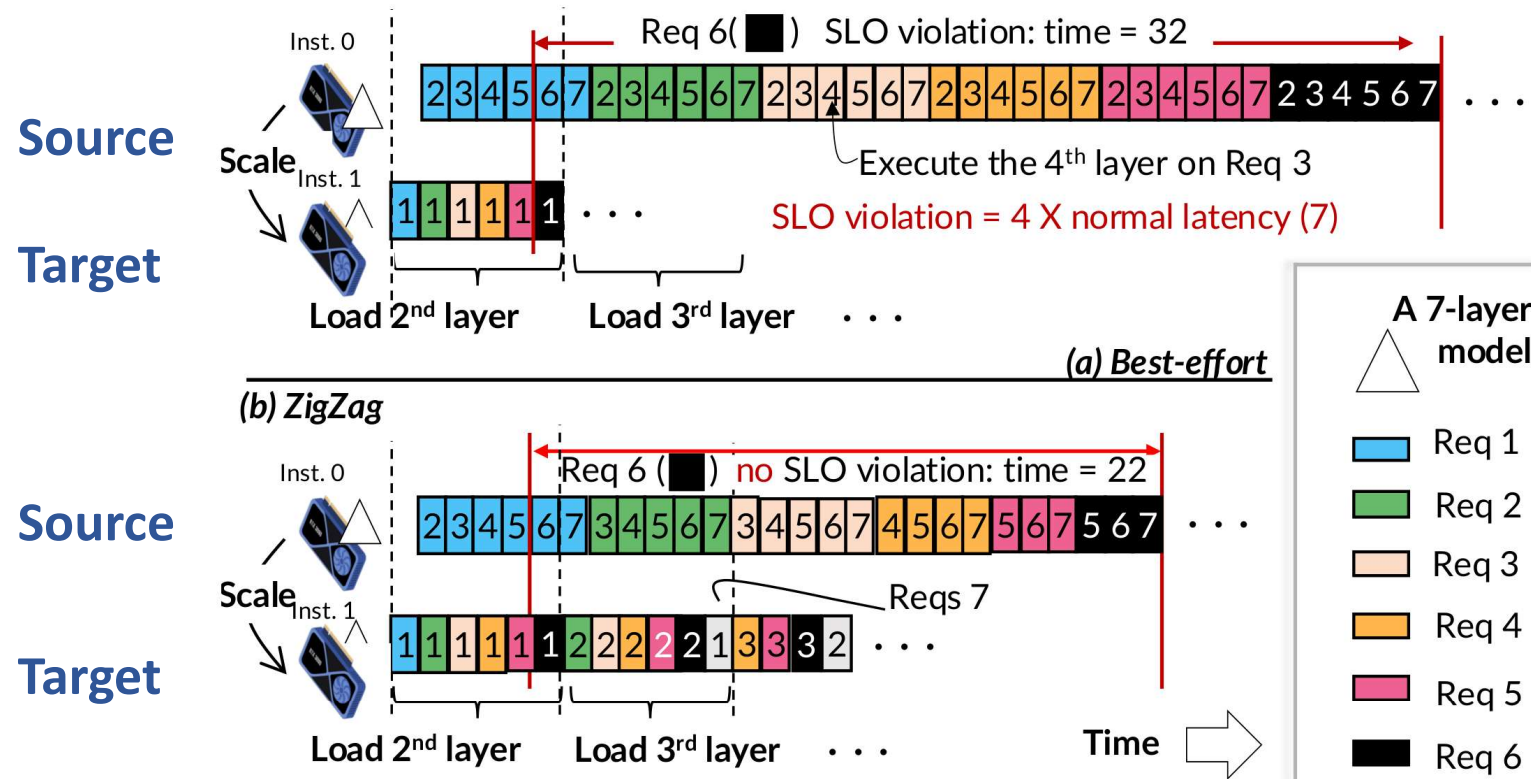


Figure 12: An illustration the necessity of zigzag scheduling. Note that the execution starts when the first layer has been loaded to instance 1 (inst.1). Our example assumes the time of loading a layer can do 6-layer computations.

Zigzag scheduling: Pipeline configuration

- Minimizing the objective requires meeting the following constraints:

S_i : numbers of layers for req i to be executed on source GPU

T_i : numbers of layers for req i to be executed on target GPU

L : Total amount of layers

N : Batch size

$Time_l$: the time to load one layer normalized to the execution time of one pipeline layer.

- utilize the waiting for the target instance to load the first layer to solve the ILP

$$Latency_{avg} = \left(\sum_{req=1}^N \sum_{i=1}^{req} S_i \right) / N$$

$$\min \quad Latency_{avg}$$

$$\text{s.t.} \quad S_i + T_i = L, \quad \forall i \quad \text{Pipeline limit (C1),}$$

$$\sum_j^i T_j \leq \sum_j^{i-1} S_j, \quad \forall i > 1 \quad \text{Pipeline dependency (C2),}$$

$$Time_l * T_i \leq \sum_j^{i-1} T_j, \quad \forall i > 1 \quad \text{Load limit(C3)}$$

Zigzag scheduling: Scheduling requests

- To achieve FIFO, we use a **priority queue** for all requests
- Whenever a new layer loads, we scan the **pending queue** and execute all requests that require that layer.
- Coping with LLM
 - the **execution time** of equal-sized batches varies
 - add a **regulation parameter** for each request
 - **live scaling a decode instance** in P/D disaggregation is impossible due to the incast nature of both parameter load and KVCache transfer
 - switch a **prefill instance** to a decode instance
 - **Scale multiple prefill** instances to compensate for the loss

```
class Scheduler: 新 *  
    def __init__(self): 新 *  
        self.ready_queue = PriorityQueue()  
        self.pending_queue = queue.Queue()
```

Global parameter pool, monitor and others

- Global parameter pool and local memory cache
 - tracks the locations of the model parameters with local memory cache.
 - only needs to ensure that each hosted model has at least one parameter copy stored either in a instance or at one of the local cache of a GPU server
- Workload monitor and scale up trigger.
 - records the serving loads with tokens per second and KVCache usage globally
 - When the average load surpasses a pre-defined upper bound, we allocate sufficient instances to meet that demand.
- Scale down.
 - when the average load falls below a lower bound in a time window, we shutdown some instances and revoke all GPUs assigned to them.

Global parameter pool, monitor and others

- Point-to-point (P2P) connection pool.
 - establishing communication group between machines is slow when using off-the-shelf group communicators (e.g., NCCL)
 - precreate a connection pool that supports full-mesh connections on each
- Fault tolerance
 - If failure occurs, auto scale instances
 - **redistribute** these parameters to other machines

Experimental Setup

• Models:

- Llama-2 7B, Llama-2 70 B , Llama-3 8B

• Baselines:

- ServerlessLLM, AllCache, DistServe

• Traces:

- BurstGPT, AzureCode, AzureConv

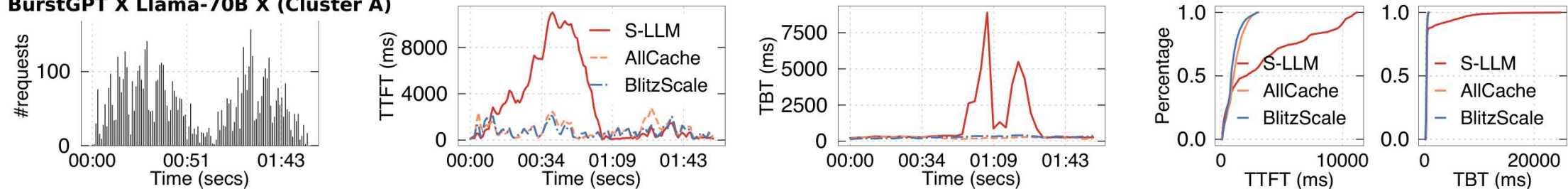
the optimal version of ServerlessLLM that always load the parameters from the host cache.

	Cluster A ($m \times g$)	Cluster B ($m \times g$)
GPU	A800 80 GB (4x8)	A100 80 GB (2x8)
GPU-GPU (intra)	1.6 Tbps NVLink	256 Gbps PCIe
GPU-GPU (inter)	100 Gbps RDMA	100 Gbps RDMA
Host-GPU	128 Gbps PCIe	128 Gbps PCIe
SSD-GPU	10 Gbps	10 Gbps

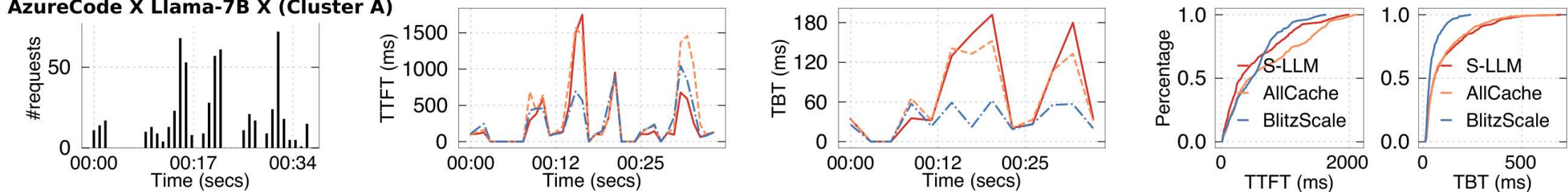
Table 1: Evaluation clusters. m is the number of hosts and g is the number of GPUs per host.

Results

BurstGPT X Llama-70B X (Cluster A)



AzureCode X Llama-7B X (Cluster A)



AzureConv X Llama3-8B X (Cluster B)

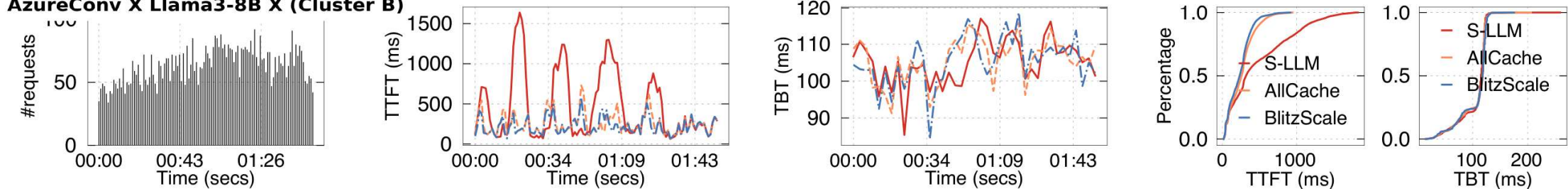


Figure 13: End-to-end performance comparison between BLITZSCALE and ServerlessLLM on various workloads, models and clusters.

Results

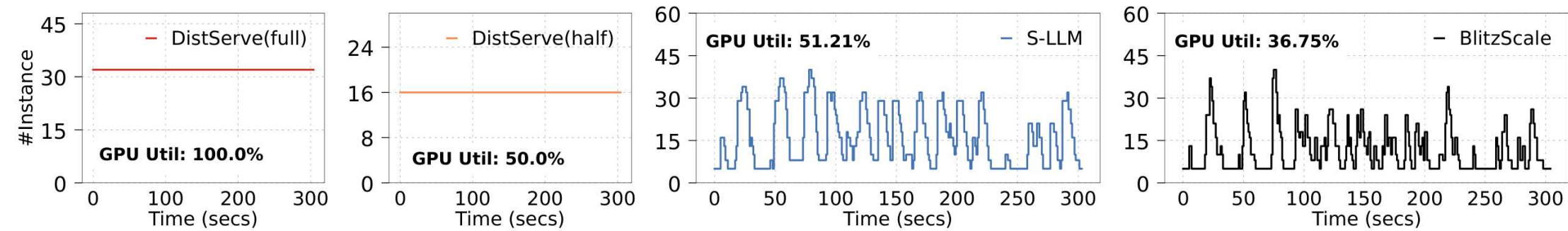


Figure 14: A comparison between GPU usage under AzureCode with Llama-2 7B.

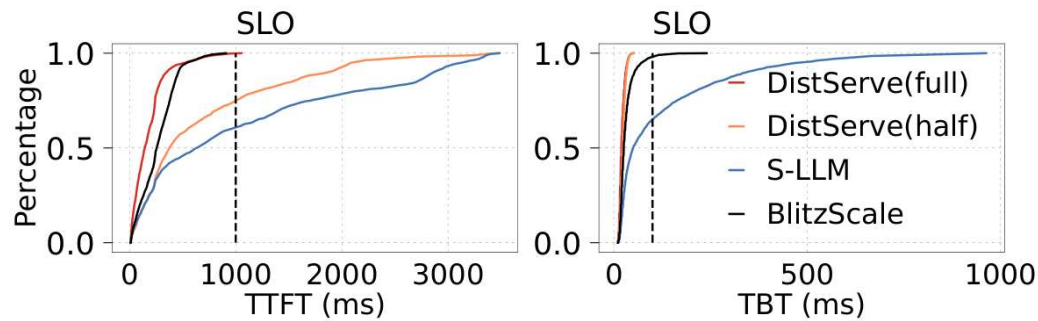


Figure 15: A comparison of TTFT and TBT under AzureCode on Llama-2 7B.

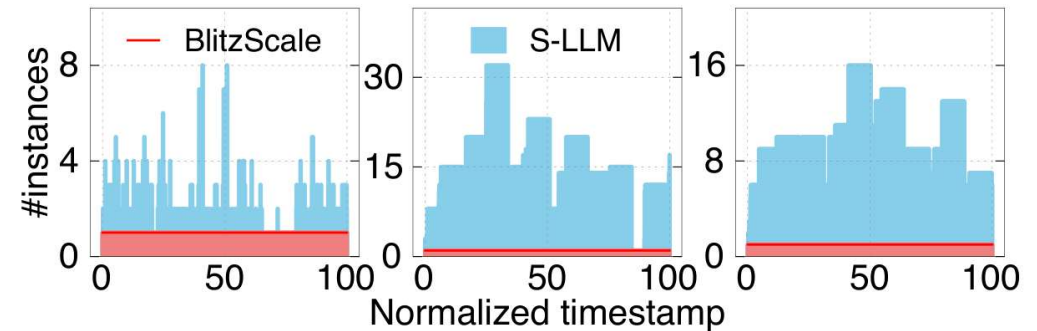


Figure 16: A comparison of host cache usage on S-LLM and BLITZSCALE under all three workloads.

Results

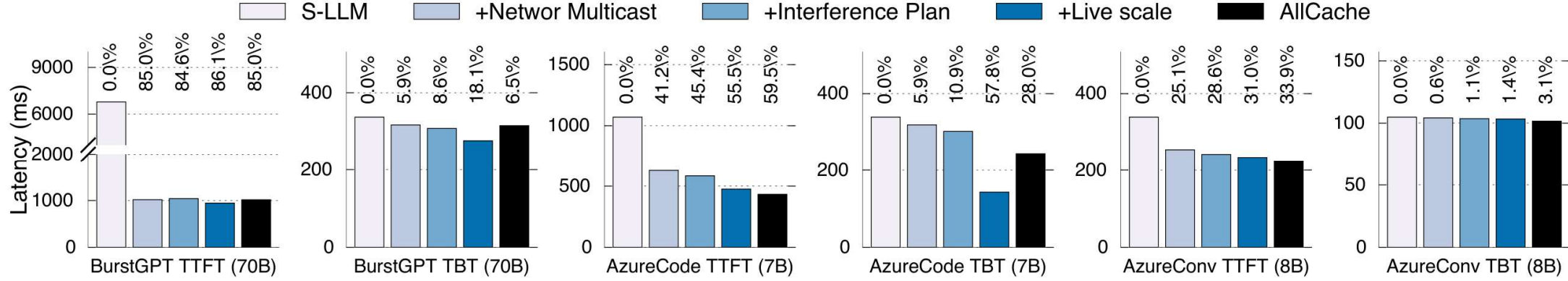


Figure 17: An ablation study on the effectiveness of our proposed techniques. All the metrics reported are average.

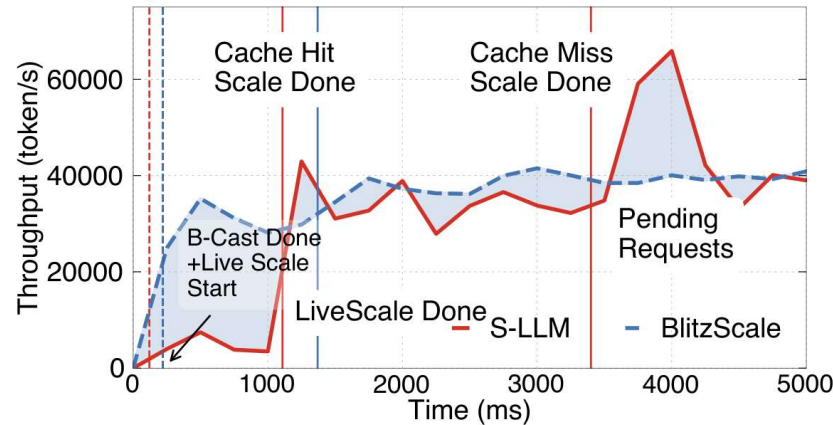


Figure 18: A detailed look how BLITZSCALE and S-LLM scale a Llama 2 7B model from one cache copy to 16 GPU instances on cluster A.

Summary

Problem

- slow and stop-the-world autoscaling

Solution

- leveraging network-based model-aware multicast
- breaking the scaling abstraction from instance-level to more fine-grained layer-level

Result

- fewer tail latency and lower TTFT, TBT